



EMBARCADERO CONFERENCE 2025

A JORNADA DO HERÓI | 30 ANOS DE
INOVAÇÃO



Delphi + RTTI: O caminho para um CRUD genérico e elegante

| Luiz Eduardo P. L. da Silveira

Formado em Sistemas de Informação pela UNESC (Centro Universitário do Espírito Santo)

Primeiro código Delphi ainda na faculdade, mas fui me apaixonar por ele só no início de 2022



Missão

Criar uma estrutura de classes que permita criar uma relação entre uma classe do Delphi com uma tabela no banco de dados para possamos criar e executar comandos SQL no SQLite e Firebird



Custom Attributes

Utilizaremos uma classe base do delphi que nos permite criar metadados e relacionar os campos de uma tabela com uma classe dentro do delphi

Exemplo Prático

```
TField = class(TCustomAttribute)
private
    FName: string;
    FSize: string;
    FDefault: string;
public
    property Name: string read FName write FName;
    property Size: string read FSize write FSize;
    property DefaultValue: string read FDefault...

    constructor Create(AName: string; ASize: string = "";
        ADefaultValue: string = "");
```

Relacionamento

```
[TField('Id'), PK, AutoInc]
property Id: Integer read FId write FId;
[TField('Description', '75'), NotNull]
property Description: string read FDescription...
[TField('Bill_Value', '15,2'), NotNull]
property Value: Currency read FValue write FValue;
```

Mas o que é o RTTI?

O RTTI (Run-Time Type Information) é uma poderosa biblioteca no delphi que nos permite acessar informações de uma classe/objeto em tempo de execução. Sua utilização nos permite:



Métodos

Executar métodos de classes em tempo de execução



Nome de propriedades

Identificar o nome de propriedades de um objeto



Valores de propriedade

Ler e verificar dinamicamente os valores de um objeto



Tipo de Dado

Identificar o tipo de dado das propriedades de um objeto (integer, float, TDatetime)



A biblioteca do RTTI pode ser acessada utilizando a uses **System.RTTI**

Como vamos utilizá-lo?



Buscar o valor de uma propriedade



Buscar os valores dos TCustomAttributes relacionados na classe



Pegar os valores da query e incluir nas propriedades da classe.



Incluir um valor na propriedade marcada como PK na classe.



A Classe RTTI é uma interface genérica `IRTTI<T: class>`

Primeiros passos com RTTI

O primeiro e importante passo a ser dado é definir o tipo da classe genérica do nosso contexto.



ClassType.GetProperties

Buscando o tipo da classe genérica

```
function TSrORMRTTI<T>.GetClassType: TRttiType;  
begin  
  var Context: TRttiContext;  
  Result := Context.GetType(TypeInfo(T));  
end;
```



Buscando o valor de uma propriedade

Com o nosso tipo definido, precisamos buscar as informações das nossas propriedades



Prop.GetValue

```
case Prop.PropertyType.TypeKind of
  tkInteger, tkEnumeration:
    Result := Prop.GetValue(TObject(AObj)).ToString;
  tkFloat:
    begin
      if Prop.PropertyType.Handle = TypeInfo(TDateTime) then
        Result := QuotedStr(Prop.GetValue(TObject(AObj)).ToString)
      else
        Result := Prop.GetValue(TObject(AObj)).ToString.Replace(',',
        ');
    end;
  ...
```



Buscar os valores dos TCustomAttributes relacionados na classe

Com o nosso tipo definido, precisamos buscar e guardar as informações dos custom attributes das nossas propriedades



ClassType.GetAttributes



Prop.GetAttributes

```
if Attribute is TTable then
begin
  ATable.Name := TTable(Attribute).Name;
  ATable.DefaultNameFields := TTable(Attribute).DefaultNameFields;
end
```

```
if Attribute is TField then
begin
  Field.Name := TField(Attribute).Name;
  Field.Size := TField(Attribute).Size;
  Field.DefaultValue := TField(Attribute).DefaultValue;
end;
```

```
Field.TType := AProp.PropertyType.TypeKind;
Field.Handle := AProp.PropertyType.Handle;
```

```
if Attribute is PK then
  Field.PK := Attribute is PK;
```



Todas as informações de atributos serão guardadas em interfaces auxiliares **ITable** e **IField**

Buscando valores retornados na query e relacionando com as propriedades do nosso objeto

Com a informação das propriedades da nossa classe, podemos percorrer o array para setar valores.



Prop.SetValue

```
case ATypeKing of
  tkFloat:
  begin
    if APropType.Handle = TypeInfo(TDateTime) then
      begin
        Aux:= AQry.FieldByName(AName).AsString;
        Result := TValue.From<TDateTime>(StrToDateTime(Aux));
      end
    else
      begin
        Aux:= AQry.FieldByName(AName).AsFloat;
        Result := TValue.From<Currency>(Aux);
      end;
    end;
  end;
```



RTTI precisará criar objetos para retornar os dados em suas propriedades, então **TRTTI<T: class, constructor>**

Estrutura das classes que criarão os comandos a serem executados



QueryBuilder que
receberá o comando
e enviará para
classe controladora



Controlador do
banco de dados
utilizado



Factory que retorna
instância pelo
DriveName



Classe para banco
utilizado



Tratamentos específicos para cada banco de dados

Precisamos tratar as diferenças de sintaxes dos bancos de forma separada.



Tipos de dados

Precisamos retornar o tipo de dado que o banco de dados aceita.



Propriedades

As propriedades dos campos no banco podem ter sintaxe diferentes entre SGDBs

Tipo de Dados

```
tkInteger,  
tkEnumeration, tkInt64:  
begin  
  if AHandle = TypeInfo(Boolean) then  
    Result := ' BOOLEAN '  
  
  if Result.Trim.IsEmpty then  
    Result := ' INTEGER '
```

Propriedades dos campos

```
if AFieldData.PK then  
begin  
  Result := cNotNull + cPK;  
  
  if AFieldData.AutoInc then  
    Result := cAutoInc_FB + cPK;  
end;
```

Utilização de constantes para criação dos comandos dinâmicos

Vamos criar uma unit contendo constantes que irão nos auxiliar com a criação dos comandos



Consts

Incluiremos os valores através do format

Existem parte de comandos SQL que podem sofrer mudanças a depender do banco que se esteja sendo executado, por exemplo, criação de FK difere entre SQLite e Firebird e por isso temos constantes diferentes para cada um

constantes

cCREATETABLE= ' CREATE TABLE %s (%s)';

cALTERTABLE = ' ALTER TABLE ';

cDROPCOLUMN = ' DROP COLUMN ';

cSELECT = ' SELECT %s FROM %s %s %s %s';

cWHERE = ' WHERE ';

cOrderBy = ' Order By ';

cEQUALS = ' = ';

cINSERT = ' INSERT INTO %s (%s) VALUES (%s)';

cUPDATEByPK = ' UPDATE %s SET %s WHERE %s ';

cDELETE = ' DELETE FROM %s WHERE %s ';

cMAX_PK = ' SELECT MAX(%s) AS PK FROM %s ';

cSeparator = '<,>';

Criando o comando a ser executado

Com os passos anteriores definidos, basta que seja criado cada método que retorna a string com o SQL do comando do contexto

Create Table

```
for Field in AFieldsData do
begin
    SQLFields := SQLFields.Replace(cSeparator, ', ');

    SQLFields := SQLFields + Field.Name + ' ' + ReturnFieldDBType(Field.TType,
    Field.Handle, Field);
    PropField := GetPropertiesField(Field);
    SQLFields := Concat(SQLFields, PropField);

    SQLFields := Concat(SQLFields, cSeparator);
end;

SQLFields := SQLFields.Replace(cSeparator, EmptyStr);
SQLFields := Concat(SQLFields, cSeparator);
SQLFields := Concat(SQLFields, GetFksFields(ATableData, AFieldsData));
SQLFields := SQLFields.Replace(cSeparator, EmptyStr);

SQL := Concat(SQL, SQLFields);

Result := Format(cCREATETABLE, [ATableData.Name, SQL]);
```


Criando a classe que executará os comandos SQL

Com a classe RTTI finalizada, podemos criar a classe que será responsável apenas por executar comandos SQL



Comando SQL

Somente precisamos receber um comando e executar, o processamento dos dados fica a cargo do RTTI



RTTI

A depender do tipo do comando utilizado, o RTTI será acionado.

```
if AComand in ESelectComands then
  Qry.Open
else
  Qry.ExecSQL;

if FConnection.InTransaction then
  FConnection.Commit;

if (AComand in ESelectComands) and (not Qry.IsEmpty) then
  ProcessQryResult(Qry, ATableData, AFieldsData, AObj, AList);

if AComand = qclINSERT then
  SetPKValue(Qry, ATableData, AFieldsData, AObj)
```



A Classe de DAO é uma interface **IDAO<T: class>**

Estrutura das classes externalizadas



Controller

Que receberá a conexão e a classe relacionada



DDL

Conterá a chamada da criação dos comandos DDL e suas validações



DML

Conterá a chamada da criação dos comandos DML e suas validações



Firedac como seu amigo na busca de metadados das tabelas

Em alguns comandos SQL, precisaremos de informações da tabela e é nesse momento que o nosso “amigo” entra



TFDMetaInfoQuery

Com ajuda do **TFDMetaInfoQuery**, podemos identificar várias informações de uma tabela e é com ele que vamos validar se a tabela existe e se existe alguma coluna que ainda não foi adicionada na tabela

Firedac o salvador da pátria

```
FDMetaInfo := TFDMetaInfoQuery.Create(nil);  
try  
  FDMetaInfo.Connection := AConnection;  
  FDMetaInfo.MetaInfoKind := mkTableFields;  
  FDMetaInfo.ObjectName := ATableName;  
  
  FDMetaInfo.Open;
```

```
FDMetaInfo := TFDMetaInfoQuery.Create(nil);  
try  
  FDMetaInfo.Connection := AConnection;  
  FDMetaInfo.MetaInfoKind := mkTables;  
  FDMetaInfo.ObjectName := ATableName;
```

- Q&A
- Perguntas





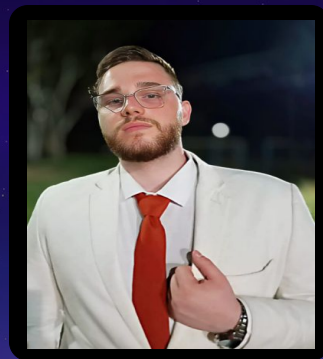
EMBARCADERO CONFERENCE 2025

A JORNADA DO HERÓI | 30 ANOS DE INOVAÇÃO



Quer me ver na
#ECON26?

Acesse o QRCode pelo
APP e avalie minha
palestra!



Luiz Eduardo P. L. Silveira



@luizpolezi



<https://www.linkedin.com/in/luiz-eduardo-pe-reira-lacerda-da-silveira-892986190>



lpolezi98@gmail.com



(27) 9 9921-7124